

Algorithmique et programmation pour l'ingénieur

TP4 — Optimisation mémoire du vecteur et généralisation du type stocké

Introduction

Dans le TP2, vous avez développé une première version d'un tableau dynamique permettant de stocker des valeurs de type `double`.

Dans le TP3, vous avez mis en place des outils de mesure afin d'évaluer expérimentalement les performances de cette structure. Vous avez notamment comparé plusieurs scénarios d'utilisation : insertions, suppressions, accès aléatoires, accès séquentiels et tri.

Dans ce TP, vous allez faire évoluer votre bibliothèque `vector` selon deux axes complémentaires :

- améliorer la gestion mémoire afin de limiter les réallocations coûteuses ;
- généraliser le type des éléments stockés afin de ne plus être limité au type `double`.

Ces évolutions seront réalisées en conservant plusieurs versions du code grâce à la compilation conditionnelle, puis en réutilisant les benchmarks du TP3 pour mesurer l'impact des choix d'implémentation.

1 Objectifs du TP

À la fin de ce TP, vous devrez être capables de :

- expliquer la différence entre `size` et `capacity` ;
- modifier une structure C en conservant plusieurs versions grâce à la compilation conditionnelle ;
- implémenter une stratégie de redimensionnement automatique ;
- écrire des tests unitaires pour vérifier le comportement de la capacité ;
- comparer expérimentalement les performances de plusieurs versions du vecteur ;
- utiliser `void *`, `elem_size`, `memcpy` et `memmove` pour rendre une structure de données générique ;
- comprendre pourquoi les pointeurs de fonctions deviennent utiles pour gérer des types contenant eux-mêmes des ressources dynamiques.

Versions utilisées dans ce TP

Dans ce TP, les versions du code sont les suivantes :

- **version par défaut** : version naïve du vecteur, spécialisée pour le type `double` ;
- **VERSION=2** : version optimisée avec gestion de la capacité, toujours spécialisée pour le type `double` ;
- **VERSION=3** : version générique du vecteur, utilisant `void *` et `elem_size`.

La version naïve correspond donc à la compilation sans définir `VERSION`.

2 Stratégie de capacité

Actuellement, dans la version naïve de votre `vector`, chaque insertion ou suppression peut entraîner une réallocation mémoire. Cette stratégie est simple à implémenter, mais elle peut devenir très coûteuse lorsque le nombre d'opérations augmente.

L'objectif de cette première partie est d'introduire une stratégie de capacité similaire à celle utilisée dans de nombreux tableaux dynamiques.

2.1 Différence entre size et capacity

Nous allons distinguer deux notions :

- **size** : nombre d'éléments effectivement stockés dans le vecteur ;
- **capacity** : nombre maximal d'éléments que le bloc mémoire actuellement alloué peut contenir sans nouvelle allocation.

On a donc toujours :

$$\text{size} \leq \text{capacity}$$

Exemple

Si un vecteur contient 10 éléments mais possède une capacité de 16, il est possible d'ajouter encore 6 éléments sans réallouer de mémoire.

2.2 Règles de redimensionnement

Dans ce TP, vous appliquerez les règles suivantes :

1. la capacité minimale est fixée à 16 éléments ;
2. à l'allocation, la capacité est initialisée à la plus petite puissance de deux supérieure ou égale à n , avec un minimum de 16 ;
3. avant une insertion, si `size == capacity`, la capacité est doublée ;
4. après une suppression, si `size <= capacity / 4`, la capacité est divisée par deux ;
5. la capacité ne descend jamais sous 16.

Calcul de la puissance de deux supérieure

Pour calculer la capacité initiale, vous pouvez utiliser l'algorithme suivant :

Entrées : Un entier n

Output : La plus petite puissance de deux supérieure ou égale à n , avec un minimum de 16

$p \leftarrow 16$;

tant que $p < n$ **faire**

$p \leftarrow 2 \times p$;

fin

retourner p ;

Algorithme 1 : Calcul de la capacité initiale

Cas particulier : allocation d'un vecteur vide

Même si $n == 0$, la version optimisée doit initialiser une capacité minimale de 16 et allouer le bloc mémoire correspondant.

Ainsi, un vecteur vide peut recevoir des éléments avec `vector_push_back()` sans devoir traiter séparément le cas `data == NULL`.

3 Compilation conditionnelle

Dans le TP3, vous avez découvert la compilation conditionnelle avec le fichier `conditional_compilation`.

Dans ce TP, ce mécanisme est utilisé pour faire cohabiter plusieurs implémentations du vecteur dans les mêmes fichiers :

- la version naïve, utilisée lorsque `VERSION` n'est pas définie ;
- la version optimisée avec `capacity`, utilisée lorsque `VERSION=2` ;
- la version générique, utilisée lorsque `VERSION=3`.

```
1 #ifndef VERSION
2     // Version naïve : vector specialise pour double
3 #elif VERSION == 2
4     // Version optimisee avec capacity
5 #elif VERSION == 3
6     // Version generique
7 #endif
```

Pourquoi utiliser la compilation conditionnelle ?

La compilation conditionnelle permet de comparer plusieurs implémentations sans dupliquer les fichiers sources.

Les programmes de test et de benchmark peuvent rester très proches : seule l'option de compilation change.

Écriture robuste des conditions

Dans les exemples de code, on utilisera de préférence :

```
1 #if defined(VERSION) && VERSION >= 2
2     // Code utilise pour les versions 2 et suivantes
3 #endif
```

Cette écriture évite toute ambiguïté lorsque la constante `VERSION` n'est pas définie.

3.1. Compilez successivement les différentes versions du projet :

(a) Version naïve :

```
make
```

(b) Version optimisée :

```
make VERSION=2
```

(c) Version générique :

```
make VERSION=3
```

3.2. Vérifiez les exécutables générés dans le répertoire `./bin`.

4 Implémentation de la version optimisée

Dans cette partie, vous allez modifier `vector.h` et `vector.c` pour ajouter la gestion de la capacité en `VERSION=2`.

4.1 Modification de la structure

- 4.1. Dans `vector.h`, utilisez la compilation conditionnelle pour ajouter un champ `capacity` uniquement lorsque `VERSION >= 2`.

4.2 Allocation et libération

- 4.1. Dans `vector.c`, mettez à jour la fonction `vector_alloc()`.

En version optimisée :

- la capacité doit être initialisée à la plus petite puissance de deux supérieure ou égale à `n`, avec un minimum de 16 ;
- le tableau `data` doit être alloué avec une taille égale à `capacity` ;
- les `size` premiers éléments doivent être initialisés à 0.0.

- 4.2. Vérifiez que `vector_free()` libère correctement la mémoire et met le pointeur à `NULL`.

4.3 Redimensionnement automatique

Pour éviter de dupliquer du code, vous allez ajouter une fonction statique dans `vector.c`.

Cette fonction n'est pas visible depuis les autres fichiers. Elle sert uniquement à l'implémentation interne du module `vector`.

- 4.1. Implémentez la fonction statique suivante :

```
1 static VectorStatus update_capacity(p_s_vector v);
```

Elle doit :

- retourner `VECTOR_ERROR_NULL_POINTER` si `v == NULL` ;
- doubler la capacité si `size == capacity` ;
- réduire la capacité de moitié si `size <= capacity / 4` et si `capacity > 16` ;
- ne jamais descendre sous une capacité de 16 ;
- retourner `VECTOR_ERROR_ALLOCATION` si un `realloc` échoue.

- 4.2. Modifiez `vector_insert()` pour utiliser `update_capacity()` en version optimisée.

- 4.3. Modifiez `vector_erase()` pour utiliser `update_capacity()` après la suppression en version optimisée.

5 Tests unitaires de la capacité

Les tests unitaires doivent vérifier que la capacité évolue correctement en version optimisée.

Attention

Le champ `capacity` n'existe pas dans la version naïve. Les tests qui accèdent directement à ce champ doivent donc être protégés par de la compilation conditionnelle.

- 5.1. Ajoutez un test de croissance automatique de la capacité.
- 5.2. Ajoutez un test de réduction automatique de la capacité.
- 5.3. Ajoutez un test de stabilité de la capacité lorsqu'aucun redimensionnement n'est nécessaire.
- 5.4. Ajoutez ces tests dans la fonction `main()` de `test_unit.c`, puis vérifiez :
- (a) que les tests passent avec la version naïve :

```
make
./bin/test_unit
```

(b) que les tests passent avec la version optimisée :

```
make VERSION=2
./bin/test_unit_v2
```

6 Évaluation expérimentale de la version optimisée

Dans cette partie, vous devez reprendre les scénarios de benchmark du TP3 afin de comparer la version naïve et la version optimisée du vecteur.

6.1. Pour chaque scénario de benchmark, comparez :

— la version naïve :

```
make
/usr/bin/time -v ./bin/bench_vector insert_erase_head 1000 100000
```

— la version optimisée :

```
make VERSION=2
/usr/bin/time -v ./bin/bench_vector_v2 insert_erase_head 1000
100000
```

Rappel sur le choix de `init_size` et `n`

Pour une taille donnée `init_size`, choisissez une valeur de `n` suffisamment grande pour obtenir un temps total de quelques secondes.

Le temps unitaire moyen d'une opération peut ensuite être estimé par :

$$\text{temps unitaire} \approx \frac{\text{temps total}}{n}$$

Erreur fréquente

Ne comparez pas directement deux temps totaux obtenus avec des valeurs différentes de `n`. Pour comparer correctement les performances, il faut ramener les mesures à un temps unitaire moyen.

6.2. Réalisez les mesures pour les scénarios suivants :

- `insert_erase_random`;
- `insert_erase_head`;
- `insert_erase_tail`;
- `read_write_random`;
- `read_write_sequential`;
- `bubble_sort`.

6.3. Pour analyser la mémoire, utilisez `memcheck` et/ou `massif`.

Mesures mémoire avec Valgrind

Pour analyser la mémoire, vous pouvez utiliser deux outils différents :

- `memcheck`, qui indique notamment le nombre total d'allocations et de libérations dans le résumé final;
- `massif`, qui permet d'observer l'évolution de l'utilisation mémoire au cours de l'exécution et le pic mémoire atteint.

Ces deux outils ne donnent pas exactement les mêmes informations : `memcheck` est utile pour détecter les fuites mémoire, tandis que `massif` est plus adapté à l'analyse du profil mémoire.

```
valgrind --tool=memcheck --leak-check=full ./bin/bench_vector  
insert_erase_head 1000 100000  
valgrind --tool=memcheck --leak-check=full ./bin/bench_vector_v2  
insert_erase_head 1000 100000
```

6.4. Reportez vos résultats dans un tableau contenant au minimum :

(`test_type`, `version`, `init_size`, `n`, `temps total`, `temps unitaire`, `allocations`, `libérations`, ...)

6.5. Rédigez une courte analyse expliquant l'impact de la stratégie de capacité sur :

- le temps d'exécution ;
- le nombre de réallocations ;
- l'utilisation mémoire maximale.

7 Généralisation du type stocké

Dans les parties précédentes, le vecteur était spécialisé pour le type `double`. Dans cette partie, vous allez implémenter une version générique sous le flag `VERSION=3`.

L'objectif est de pouvoir stocker différents types de données dans un même type de structure `vector`, sans réécrire une bibliothèque différente pour chaque type.

Principe général

Pour rendre le vecteur générique, on remplace :

```
1 double *data;
```

par :

```
1 void *data;
```

```
2 size_t elem_size;
```

Le champ `elem_size` indique la taille en octets de chaque élément stocké.

Exemple

Pour stocker des entiers :

```
1 p_s_vector v = vector_alloc(0, sizeof(int));
2
3 int x = 42;
4 vector_push_back(v, &x);
5
6 int y = 0;
7 vector_get(v, 0, &y);
```

7.1 Structure et prototypes

7.1. Modifiez `vector.h` pour que la structure dépende de la version compilée.

7.2. En version générique, adaptez les prototypes des fonctions :

- `vector_alloc()` doit recevoir la taille d'un élément ;
- `vector_set()` doit recevoir un pointeur vers l'élément à copier ;
- `vector_get()` doit recevoir un pointeur vers la zone de destination ;
- `vector_insert()` et `vector_push_back()` doivent recevoir un pointeur vers l'élément à insérer.

7.2 Allocation générique

7.1. Implémentez `vector_alloc()` pour la version générique.

7.3 Accès aux éléments avec `memcpy`

Pour accéder à un élément, il faut calculer son adresse dans le bloc mémoire.

L'adresse de l'élément d'indice `i` est :

$$(\text{char}^*)v \rightarrow \text{data} + i * v \rightarrow \text{elem_size}$$

Le cast en `char *` est nécessaire car un pointeur de type `char *` avance octet par octet.

7.1. Implémentez `vector_set()` et `vector_get()` en utilisant `memcpy()`.

7.4 Insertions, suppressions et `memmove`

Lorsqu'on insère ou supprime un élément, il faut décaler des blocs mémoire. Dans ce cas, les zones source et destination peuvent se chevaucher.

`memcpy` ou `memmove` ?

La fonction `memcpy` permet de copier un bloc mémoire lorsque les zones source et destination ne se chevauchent pas.

Lorsqu'on décale des éléments dans un tableau, les zones mémoire peuvent se chevaucher. Dans ce cas, il faut utiliser `memmove`, qui est prévue pour gérer correctement les recouvrements mémoire.

7.1. Implémentez `vector_insert()` pour la version générique.

7.2. Implémentez `vector_erase()` pour la version générique.

7.3. Adaptez `vector_push_back()` et `vector_pop_back()`.

8 Adaptation des tests et des benchmarks pour VERSION=3

La version générique modifie les signatures de plusieurs fonctions. Vous devez donc adapter `test_unit.c` et `bench_vector.c` pour que les trois versions restent compilables.

8.1. Adaptez les tests unitaires pour qu'ils fonctionnent avec `VERSION=3`.

Exemple de test pour des entiers

```
1  int test_vector_generic_int() {
2  #if defined(VERSION) && VERSION >= 3
3      p_s_vector v = vector_alloc(0, sizeof(int));
4      ASSERT(v != NULL);
5
6      int x = 42;
7      ASSERT(vector_push_back(v, &x) == VECTOR_SUCCESS);
8
9      int y = 0;
10     ASSERT(vector_get(v, 0, &y) == VECTOR_SUCCESS);
11     ASSERT(y == 42);
12
13     vector_free(&v);
14 #endif
15     return 0;
16 }
```

8.2. Donnez un exemple d'utilisation du vecteur générique pour stocker une structure :

```
1  struct point {
2      double x;
3      double y;
4  };
```

8.3. Adaptez `bench_vector.c` pour qu'il fonctionne avec la version générique.

(a) Lorsque `VERSION < 3`, conservez les appels existants :

```
1  p_s_vector v = vector_alloc(init_size);
2  vector_push_back(v, 0.0);
```

(b) Lorsque `VERSION >= 3`, utilisez la taille des éléments :

```
1  p_s_vector v = vector_alloc(init_size, sizeof(double));
2
3  double x = 0.0;
4  vector_push_back(v, &x);
```

(c) Adaptez de la même manière les appels à `vector_set()`, `vector_get()` et `vector_insert()`.

9 Limites de la version générique et pointeurs de fonctions

La version générique proposée dans ce TP copie les éléments avec `memcpy()`. Cela signifie que les octets de l'élément sont copiés tels quels.

Copies superficielles et copies profondes

Cette approche fonctionne correctement pour :

- les types simples : `int`, `double`, `char`, etc. ;
- les structures ne contenant pas de pointeurs internes.

En revanche, si une structure contient un pointeur, par exemple :

```
1 struct student {  
2     char *name;  
3     int age;  
4 };
```

alors `memcpy()` copie uniquement l'adresse contenue dans `name`, et non la chaîne de caractères pointée. On parle alors de copie superficielle.

Pour gérer correctement ce type de données, il faudrait fournir au vecteur des fonctions spécialisées :

- une fonction de copie pour réaliser une copie profonde de chaque élément ;
- une fonction de libération pour libérer proprement les ressources internes de chaque élément.

Ces fonctions peuvent être passées au vecteur sous forme de pointeurs de fonctions.

Pointeurs de fonctions : idée générale

En C, une fonction peut être manipulée à travers son adresse. On peut donc stocker l'adresse d'une fonction dans une variable appelée pointeur de fonction.

Cela permet d'écrire du code générique : au lieu d'imposer une seule manière de copier ou de libérer un élément, on laisse l'utilisateur fournir les fonctions adaptées à son type de données.

Par exemple, une fonction de copie pourrait avoir le prototype suivant :

```
1 typedef void (*copy_fn)(void *dest, const void *src);
```

et une fonction de libération :

```
1 typedef void (*free_fn)(void *elem);
```

Dans ce TP, vous n'avez pas besoin d'implémenter complètement ce mécanisme. Vous devez cependant comprendre pourquoi il devient nécessaire lorsqu'un élément contient lui-même des ressources dynamiques.

- 9.1. Expliquez pourquoi un vecteur générique basé uniquement sur `memcpy()` n'est pas suffisant pour stocker proprement des structures contenant des pointeurs internes.
- 9.2. Proposez les prototypes de deux pointeurs de fonctions qui pourraient être utilisés pour copier et libérer correctement les éléments stockés dans un vecteur générique.

10 Comparaison expérimentale des trois versions

Vous devez maintenant comparer les performances des trois versions :

- version naïve : `make` ;
- version optimisée : `make VERSION=2` ;
- version générique : `make VERSION=3`.

- 10.1. Reprenez les benchmarks du TP3 et comparez les trois versions pour les scénarios compatibles.

```

make
/usr/bin/time -v ./bin/bench_vector read_write_random 1000 100000

make VERSION=2
/usr/bin/time -v ./bin/bench_vector_v2 read_write_random 1000 100000

make VERSION=3
/usr/bin/time -v ./bin/bench_vector_v3 read_write_random 1000 100000

```

10.2. Calculez le temps unitaire moyen pour chaque version.

10.3. Analysez les différences observées :

- la version optimisée est-elle plus rapide pour les insertions/suppressions ?
- la version générique est-elle plus coûteuse que la version spécialisée ?
- quel compromis observez-vous entre généricité, performance et flexibilité ?

11 Travail à rendre

Vous devrez rendre deux éléments distincts :

1. une archive contenant le code du projet ;
2. un rapport de synthèse regroupant le travail réalisé dans les TP2, TP3 et TP4.

Archive de code

L'archive de code devra contenir :

- les fichiers `.c` et `.h` complétés ;
- le `Makefile` ;
- le fichier `README` complété ;
- les tests unitaires ajoutés ;
- les éventuels fichiers de résultats bruts utilisés pour produire les tableaux et graphiques.

Rapport de synthèse

Le rapport ne doit pas recopier le code source, puisque celui-ci est rendu séparément dans l'archive de code. Il doit plutôt présenter et analyser le travail réalisé sur l'ensemble de la séquence TP2-TP4.

Le rapport devra contenir :

- une présentation rapide de l'implémentation naïve du vecteur réalisée au TP2 ;
- un rappel des fonctions principales développées et des choix d'implémentation retenus ;
- une description de la méthodologie expérimentale mise en place au TP3 pour évaluer les performances ;
- les tableaux de mesures obtenus lors des benchmarks ;
- au moins deux graphiques permettant de comparer les performances ;
- une comparaison entre la version naïve et la version optimisée avec `capacity` ;
- une comparaison entre la version spécialisée pour `double` et la version générique ;
- une analyse du compromis entre performance, utilisation mémoire, généricité et complexité du code ;
- une courte conclusion présentant les limites de votre implémentation et les améliorations possibles.

Contenu attendu du rapport

Le rapport doit principalement porter sur l'évaluation des performances et la comparaison des différentes implémentations. Il ne s'agit pas de recopier les fichiers `vector.c`, `vector.h` ou `bench_vector.c`, mais d'expliquer les choix réalisés, de présenter les résultats obtenus et de les interpréter.

Qualité attendue du rendu

Le code rendu doit compiler sans erreur avec :

```
make  
make VERSION=2  
make VERSION=3
```

Les tests unitaires doivent s'exécuter correctement pour les trois versions. Les fichiers doivent être organisés dans les bons répertoires, et le code doit être commenté.

Le rapport doit être structuré, lisible et centré sur l'analyse. Les tableaux et graphiques doivent être accompagnés d'une interprétation.